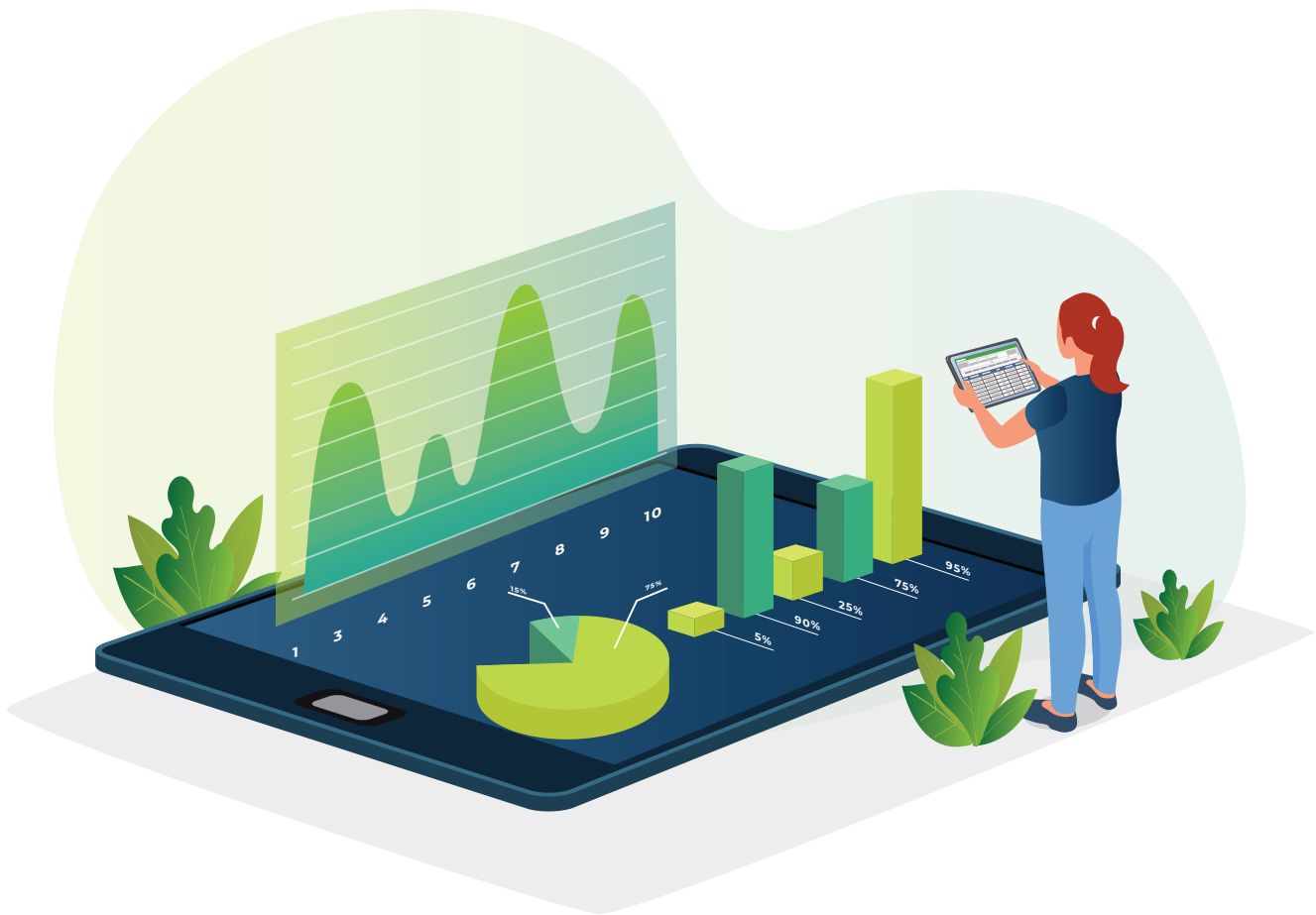




Making a dashboard interactive is crucial for engaging users and enabling them to quickly and easily interpret data through dynamic graphical representations.

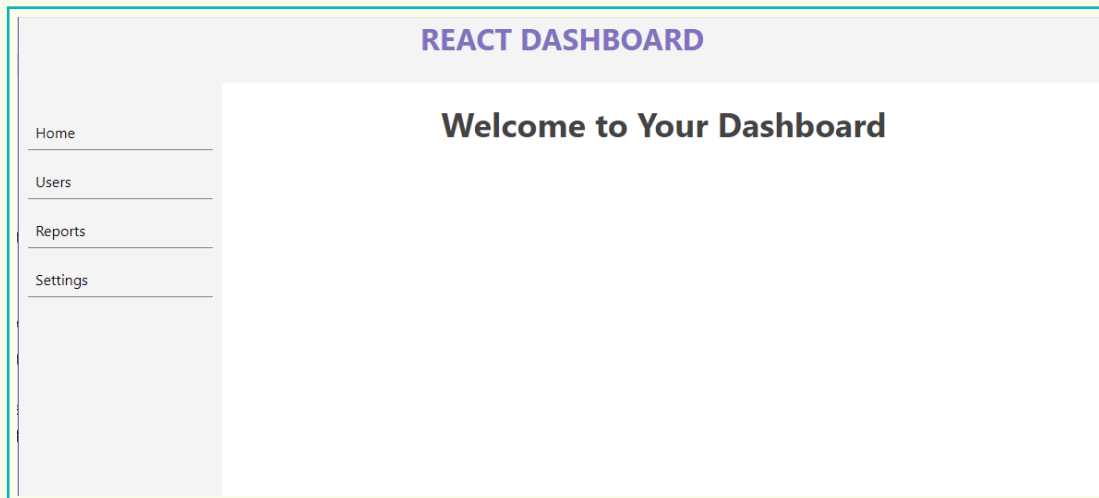


This lesson focusses on developing the existing Web Dashboard component to increase its interactivity and appeal.

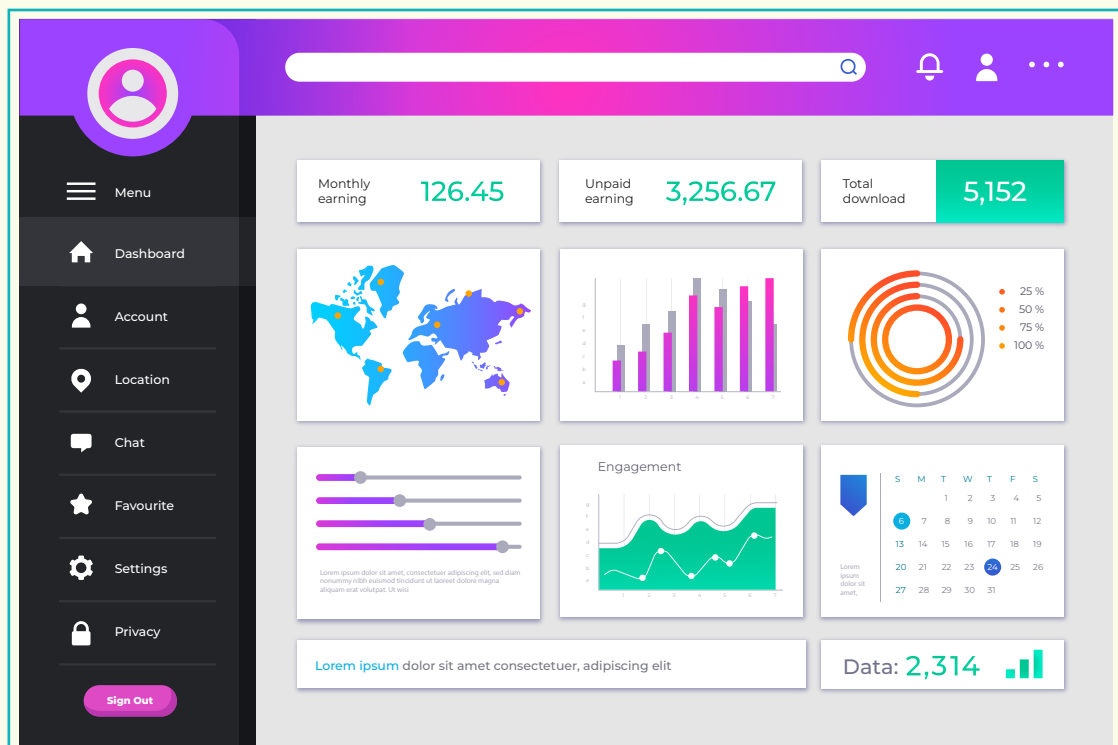
Interactive components or elements not only enhance the user experience, but also facilitate a deeper understanding of data by allowing users to explore and interact with the information in real-time.

Recall:

In the previous lesson, a basic React project was created with a few components. The *Header* component was positioned at the top of the application, the *Sidebar* component was aligned to the left, and the *Dashboard* component was positioned on the right side of the application.



The *Dashboard* component currently contains only a simple heading. This component will be updated to include widgets. **Widgets** refer to small rectangular or square-shaped elements on a dashboard that display or highlight key information, show real-time updates, or allow user interaction. They enable users to view key metrics, reports, and other information in a compact and easily accessible format.



A few use cases of Dashboards are provided below:

- **Business Intelligence Dashboards:** Monitor sales, marketing, finance, and other business metrics.
- **Project Management Dashboards:** Track project progress, deadlines, resource allocation, etc.
- **IT Monitoring Dashboards:** Monitor system health, uptime, network traffic, and server performance.
- **Customer Support Dashboards:** Track support ticket status, response times, and customer satisfaction metrics.

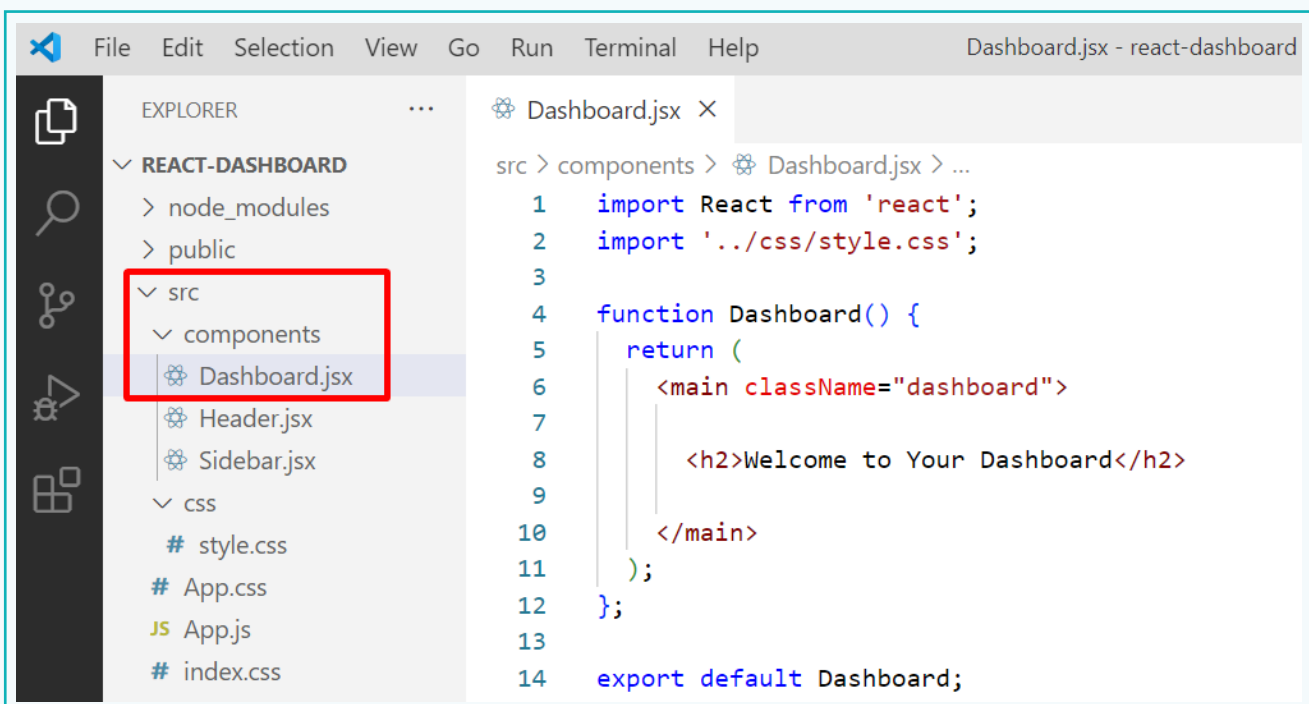
Activity

Enhance the 'Dashboard' component to improve its appearance and overall user experience.

Part 1 - Add Widgets to the Dashboard

The following activity will be executed on the existing 'react-dashboard' project from the previous lesson. To continue, open the folder containing the project on VS Code.

- 1 Open the 'Dashboard.jsx' file located in the 'components' folder.



```
File Edit Selection View Go Run Terminal Help Dashboard.jsx - react-dashboard

EXPLORER
└─ REACT-DASHBOARD
  └─ node_modules
    └─ public
      └─ src
        └─ components
          └─ Dashboard.jsx
            └─ Header.jsx
            └─ Sidebar.jsx
          └─ css
            └─ style.css
            └─ App.css
            └─ App.js
            └─ index.css

src > components > Dashboard.jsx > ...
1  import React from 'react';
2  import '../css/style.css';
3
4  function Dashboard() {
5    return (
6      <main className="dashboard">
7        <h2>Welcome to Your Dashboard</h2>
8      </main>
9    );
10 }
11
12 export default Dashboard;
```

Activity

- 2 Inside the `<main>` container, create another `<div>` container and assign a class `dashboard-widgets` to it, using the `className` attribute.

```
Dashboard.jsx X
src > components > Dashboard.jsx > ...
1  import React from 'react';
2  import '../css/style.css';
3
4  function Dashboard() {
5    return (
6      <main className="dashboard">
7        <h2>Welcome to Your Dashboard</h2>
8
9        <div className="dashboard-widgets">
10
11      </div>
12    </main>
13  );
14
```

- 3 Inside this container, add the respective widgets. Include three widgets, each with its own `<div>` tag and class `widget`.

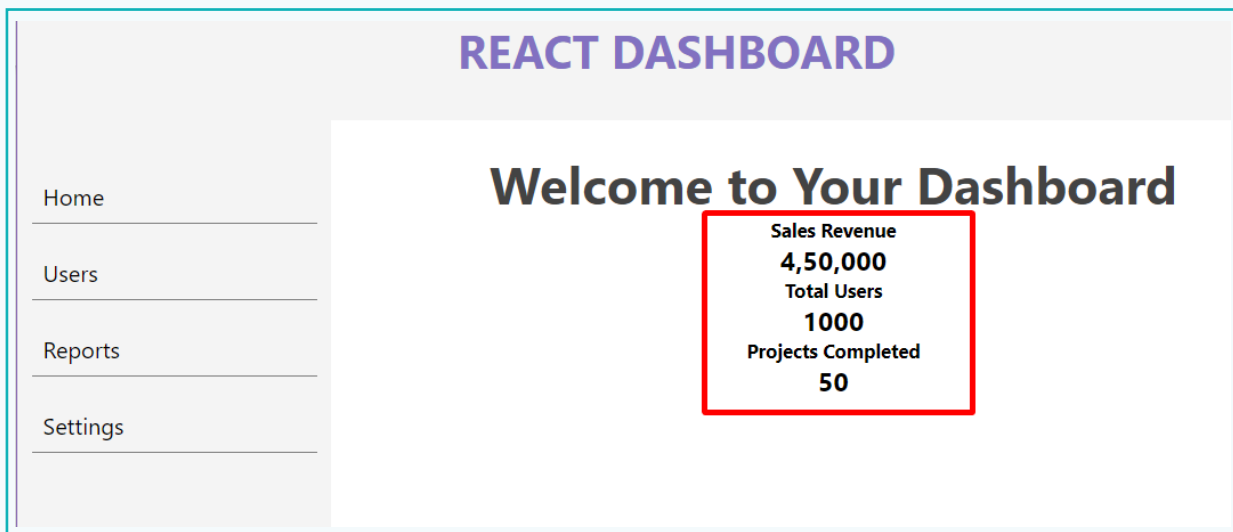
```
4  function Dashboard() {
5    return (
6      <main className="dashboard">
7        <h2>Welcome to Your Dashboard</h2>
8        <div className="dashboard-widgets">
9
10        <div className="widget">
11          <h5> Sales Revenue </h5>
12          <h3> 4,50,000</h3>
13        </div>
14
15        <div className="widget">
16          <h5> Total Users </h5>
17          <h3> 1000 </h3>
18        </div>
19
20        <div className="widget">
21          <h5> Projects Completed </h5>
22          <h3> 50 </h3>
23        </div>
24      </div>
25    </main>
26  );
27
```

- 4 Execute the application using the `npm start` command in the Terminal.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS D:\react-dashboards> npm start
```

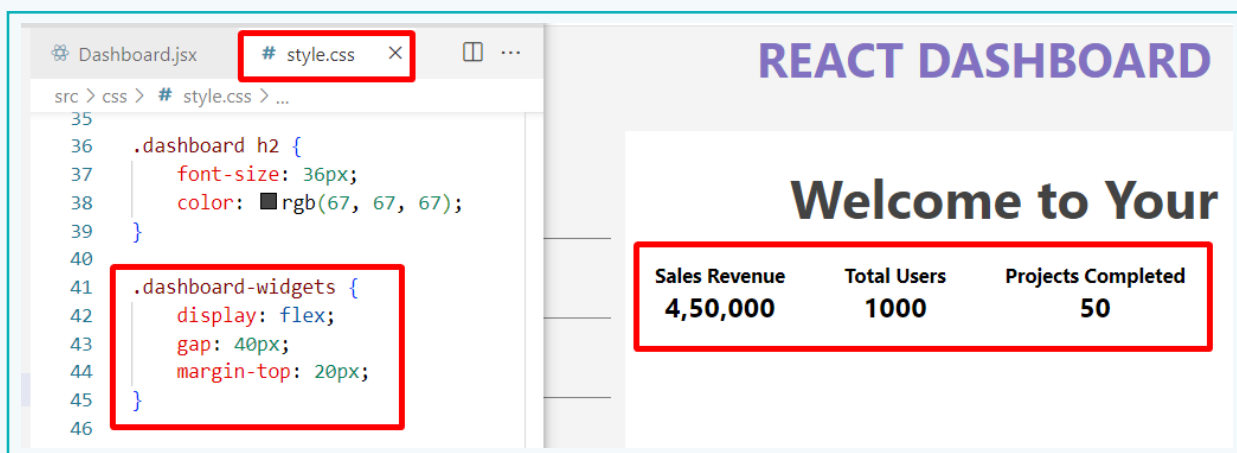
Activity

- The application will appear in the browser as shown.



Part 2 - Design the Widgets using CSS

- Open the 'style.css' file and use the CSS *display: flex* property to align the widgets horizontally. Specify the margin using the *margin-top* property and add spacing between the widgets using the *gap* property.



Activity

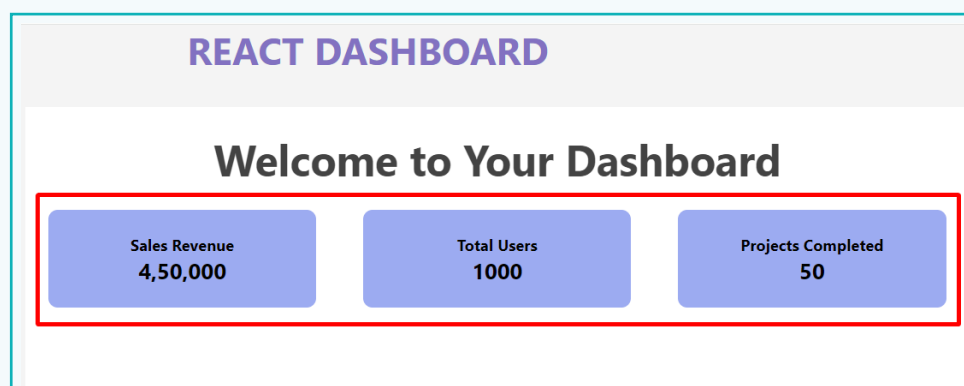
- 2 To design the individual widgets, specify a set of CSS properties for the 'widget' selector.

```
# style.css
src > css > # style.css > ...
40
41 .dashboard-widgets {
42     display: flex;
43     gap: 40px;
44     margin-top: 20px;
45 }
46
47 .widget {
48     background-color: rgb(156, 171, 241);
49     padding: 20px;
50     border-radius: 8px;
51     flex: 1;
52     text-align: center;
53 }
54
```

Note:

The `flex: 1` CSS property makes the individual widgets (flex items) expand or shrink equally, to fill up the available space.

Output after applying the CSS properties.

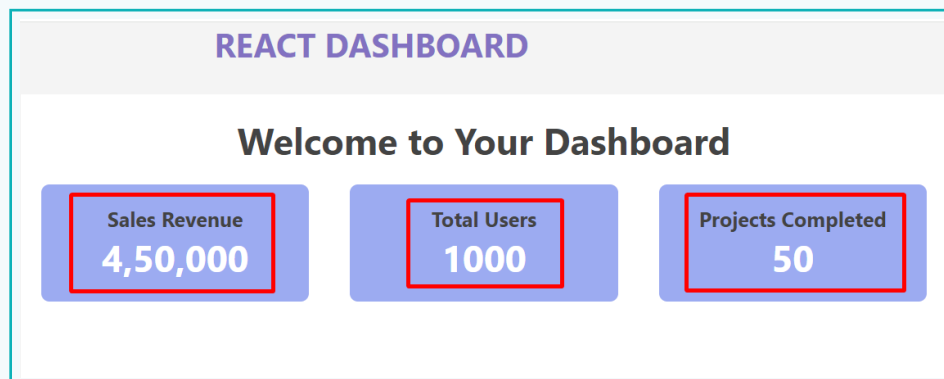


Activity

- 3 Adjust the font size and colour of the text within the widgets to enhance their appearance.

```
# style.css  X
src > css > # style.css > ...
54
55 .widget h5 {
56   font-size: 20px;
57   color: ■rgb(67, 67, 67);
58 }
59
60 .widget h3 {
61   font-size: 36px;
62   color: □white;
63 }
64
```

Output after applying the CSS properties.



Part 3 - Work with Charts in React

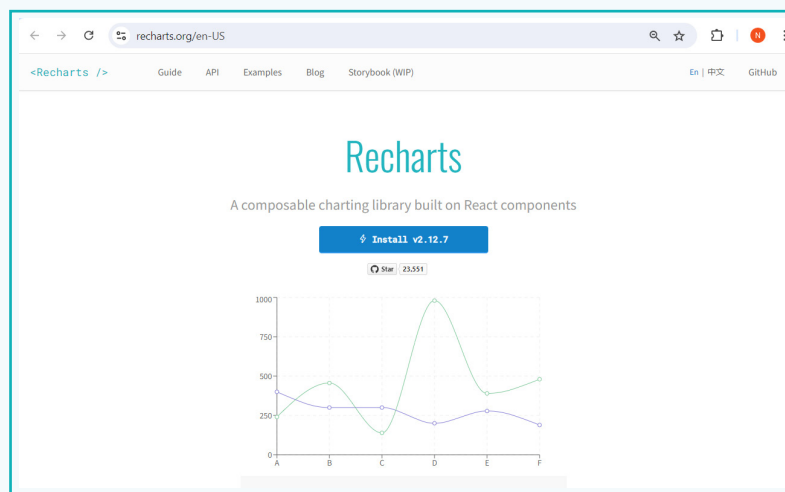
This component will be updated to include widgets. Widgets refer to small rectangular or square-shaped elements on a dashboard that display or highlight key information, show real-time updates, or allow for user interaction. They enable users to view key metrics, reports, and other information in a compact and easily accessible format.

Charts, Bar Diagrams, and other visual elements play a crucial role in Dashboards by transforming raw data into visual formats. They make Dashboards more interactive and engaging, improving the overall user experience.

Activity

Recharts is a popular charting library designed specifically for React applications. It allows the user to create customizable and interactive data visualisations easily. Integrating charts into a React application is easy using Recharts. To get a better understanding, visit the official site of Recharts - <https://recharts.org/en-US>

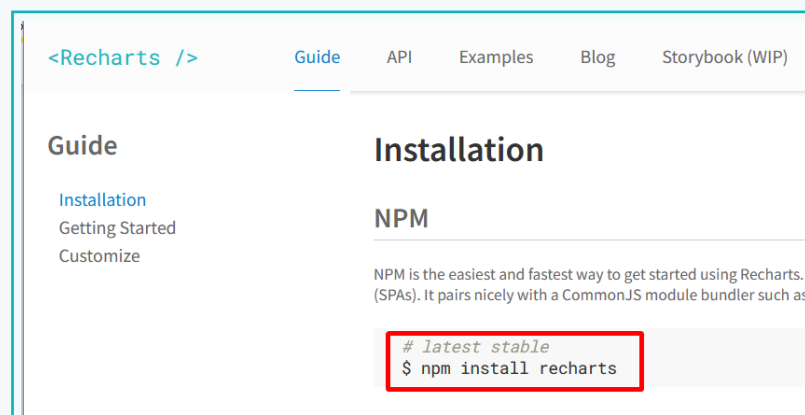
- 1 To install the library, open the website 'www.recharts.org' and click on the 'Install' button.



Bytes

Material-UI is a popular React UI framework that provides a set of reusable and customisable design components. It helps developers to build modern, responsive web applications quickly and efficiently.

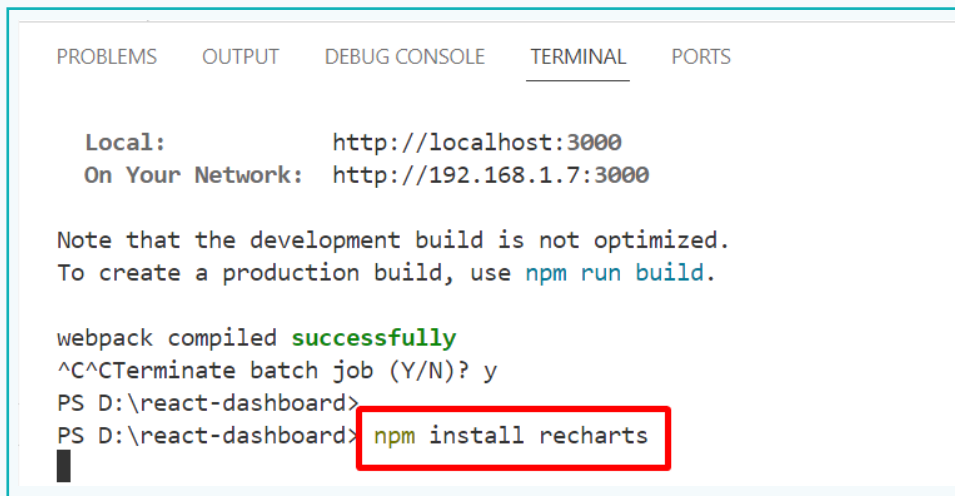
- 2 The 'Installation' page will appear, where the installation instructions are provided. Choose the 'NPM' way for installation.



Activity

- 3 Go back to VS Code and in the Terminal, press 'Ctrl + C' to stop the running application. Then, type the command `npm install recharts`.

This will download all the necessary files and install the library within a few seconds.



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Local:      http://localhost:3000
On Your Network: http://192.168.1.7:3000

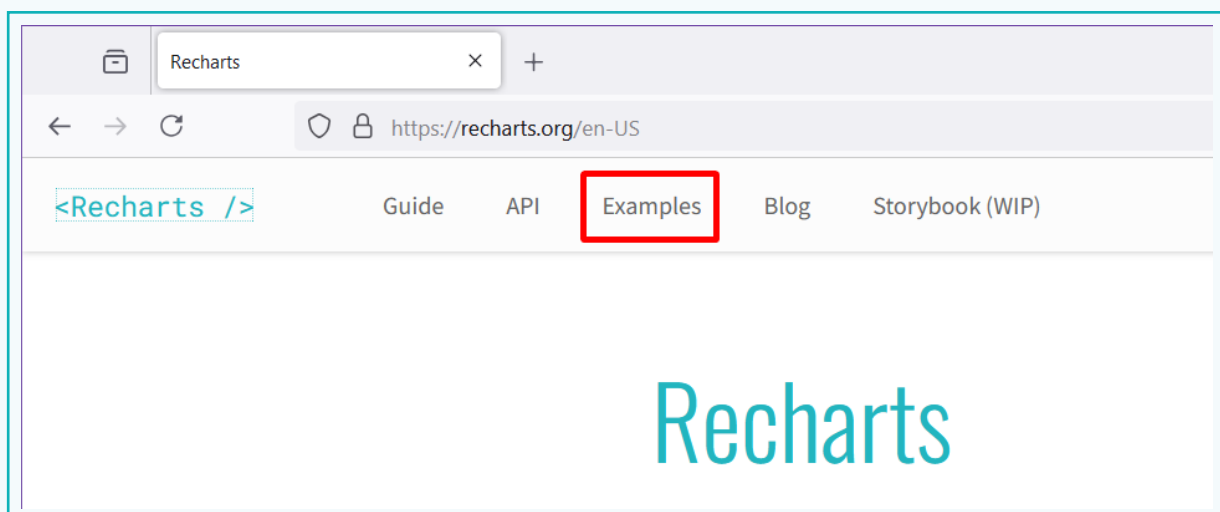
Note that the development build is not optimized.
To create a production build, use npm run build.

webpack compiled successfully
^^^CTerminate batch job (Y/N)? y
PS D:\react-dashboard>
PS D:\react-dashboard> npm install recharts
```

Part 4 - Work with Recharts

To start embedding charts into the application, follow the documentation provided by Recharts. The site offers various examples that illustrate how different visualisations can be implemented.

- 1 On the website 'www.recharts.org', click on 'Examples'.



Activity

- In the next window, examples of several types of charts and other diagrams will be displayed. Use **SimpleLineChart**. On the right-hand side, a sample Line Chart and the code responsible for generating the chart will be visible.



To use the components needed to generate the 'LineChart,' we need to import them from the 'recharts' library into the Dashboard.jsx file.

- Import the various components responsible for generating the *LineChart* by copying the second *import* statement.

```

1 import React, { PureComponent } from 'react';
2 import { LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip, Legend, ResponsiveContainer } from 'recharts';
3
4 const data = [
5   {
6     name: 'Page A',
7     uv: 4000,
8     pv: 2400,
9     amt: 2400,
10  },

```

Note:

Skip the first import statement since 'React' is already imported.

- Open the 'Dashboard.jsx' file and paste the copied *import* statement.

```

Dashboard.jsx
rc > components > Dashboard.jsx > Dashboard
1 import React from 'react';
2 import './css/style.css';
3
4 import { LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip, Legend, ResponsiveContainer } from 'recharts';
5
6 function Dashboard() {
7   return (
8     <main className="dashboard">
9       <h2>Welcome to Your Dashboard</h2>
10      <div className="dashboard-widgets">
11

```

Activity

- 5 Next, we need to specify the data for the chart. In Recharts, this data is presented in an Array format, as shown.

```
> Run
1 import React, { PureComponent } from 'react';
2 import { LineChart, Line, XAxis, YAxis, CartesianGrid
3
4 const data = [
5   {
6     name: 'Page A',
7     uv: 4000,
8     pv: 2400,
9     amt: 2400,
10  },
11  {
12    name: 'Page B',
13    uv: 3000,
14    pv: 1398,
15    amt: 2210,
16  },
17  {
18    name: 'Page C',
19    uv: 2000,
20    pv: 9800,
21    amt: 2290,
22  }
23 ];
24
```

Represents the data for the Chart

Now that the required data format is understood, replace the dummy data with the actual data.

```
src > components > Dashboard.jsx > Dashboard
1 import React from 'react';
2 import '../css/style.css';
3
4 import { LineChart, Line, XAxis, YAxis, CartesianGrid,
5
6 const salesData = [
7   {
8     month: 'Jan',
9     amt: 10000,
10  },
11  {
12    month: 'Feb',
13    amt: 14000,
14  },
15  {
16    month: 'March',
17    amt: 9000,
18  },
19  {
20    month: 'April',
21    amt: 17000,
22  }
23 ];
24
```

- 6 In the 'Dashboard.jsx' file, create an array *salesData* and create multiple objects within it. Each object will represent the total amount of products sold during a particular month.

Activity

Part 5 - Generate the Line Chart

Let us generate a chart that shows the number of sales in different months.

This involves using the *Recharts* components that were already imported. This portion of the code is responsible for generating the chart based on the provided data.

1 Copy the shown code from the website.

```

24
25 export default class Example extends PureComponent {
26   static demoUrl = 'https://codesandbox.io/p/sandbox/line-chart-width-xaxis';
27
28   render() {
29     return (
30       <ResponsiveContainer width="100%" height="100%">
31         <LineChart
32           width={500}
33           height={300}
34           data={data}
35           margin={{
36             top: 5,
37             right: 30,
38             left: 20,
39             bottom: 5,
40           }}
41         >
42           <CartesianGrid strokeDasharray="3 3" />
43           <XAxis dataKey="name" />
44           <YAxis />
45           <Tooltip />
46           <Legend />
47           <Line type="monotone" dataKey="pv" stroke="#8884d8" activeDot={{
48             r: 8 }} />
49           <Line type="monotone" dataKey="uv" stroke="#82ca9d" />
50         </LineChart>
51       </ResponsiveContainer>
52     );
53   }
54 }

```

2 In the 'Dashboard.jsx' file, paste the copied code present inside the *return* statement, just before the closing *</main>* tag.

```

src > components > Dashboard.jsx > ...
26 function Dashboard() {
41
42   <div className="widget">
43     <h5> Projects Completed </h5>
44     <h3> 50 </h3>
45   </div>
46
47   <div>
48     <ResponsiveContainer width="100%" height="100%">
49       <LineChart
50         width={500}
51         height={300}
52         data={data}
53         margin={{
54           top: 5,
55           right: 30,
56           left: 20,
57           bottom: 5,
58         }}
59       >
60         <CartesianGrid strokeDasharray="3 3" />
61         <XAxis dataKey="name" />
62         <YAxis />
63         <Tooltip />
64         <Legend />
65         <Line type="monotone" dataKey="pv" stroke="#8884d8" activeDot={{ r: 8 }} />
66         <Line type="monotone" dataKey="uv" stroke="#82ca9d" />
67       </LineChart>
68     </ResponsiveContainer>
69   </div>
70 }

```

Activity

3 The entire code within the `<ResponsiveContainer>` component is not required. Update the code as follows:

- Remove the 'width', 'height', and 'margin' attributes within the `<LineChart>` component.
- Update the 'data' value with the array `salesData`.

```
<ResponsiveContainer width="100%" height="100%">  
  <LineChart data={salesData} >  
    <CartesianGrid strokeDasharray="3 3" />  
    <XAxis dataKey="name" />  
    <YAxis />  
    <Tooltip />  
    <Legend />  
    <Line type="monotone" dataKey="pv" stroke="#8884d8" activeDot={{ r: 8 }} />  
    <Line type="monotone" dataKey="uv" stroke="#82ca9d" />  
  </LineChart>  
</ResponsiveContainer>
```

4 Update the 'dataKey' attribute within the `<XAxis />` component by changing the value from 'name' to 'month'. Here, 'month' represents the 'month' key in the `salesData` array.

```
<ResponsiveContainer width="100%" height="100%">  
  <LineChart data={salesData} >  
    <CartesianGrid strokeDasharray="3 3" />  
    <XAxis dataKey="month" />  
    <YAxis />  
    <Tooltip />  
    <Legend />  
    <Line type="monotone" dataKey="pv" stroke="#8884d8" activeDot={{ r: 8 }} />  
    <Line type="monotone" dataKey="uv" stroke="#82ca9d" />  
  </LineChart>  
</ResponsiveContainer>
```

Activity

- 5 Remove the `<Tooltip />` and `<Legend />` components since these are not required.

```
<ResponsiveContainer width="100%" height="100%">
  <LineChart data={salesData} >
    <CartesianGrid strokeDasharray="3 3" />
    <XAxis dataKey="month" />
    <YAxis />
    <Line type="monotone" dataKey="pv" stroke="#8884d8" activeDot={{ r: 8 }} />
    <Line type="monotone" dataKey="uv" stroke="#82ca9d" />
  </LineChart>
</ResponsiveContainer>

</main>
```

`<Tooltip />` and `<Legend />` component remove

- 6 Remove the components from the import statement as well.

```
Dashboard.jsx X
c > components > Dashboard.jsx > salesData
1 import React from 'react';
2 import './css/style.css';
3
4 import { LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip, Legend, ResponsiveContainer } from 'recharts';
5
6 const salesData = [
7   {
8     month: 'Jan',
9     amt: 10000,
10  },
11 ];
```

Remove from the Import statement

- 7 Remove the first `<Line>` component, as the chart will only contain a single line. In the second `<Line>` component, update the `dataKey` attribute by replacing 'uv' with 'amt'. Here, 'amt' refers to the 'amt' key present in the `salesData` array.

```
<ResponsiveContainer width="100%" height="100%">
  <LineChart data={salesData} >
    <CartesianGrid strokeDasharray="3 3" />
    <XAxis dataKey="month" />
    <YAxis />
    <Line type="monotone" dataKey="amt" stroke="#82ca9d" />
  </LineChart>
</ResponsiveContainer>

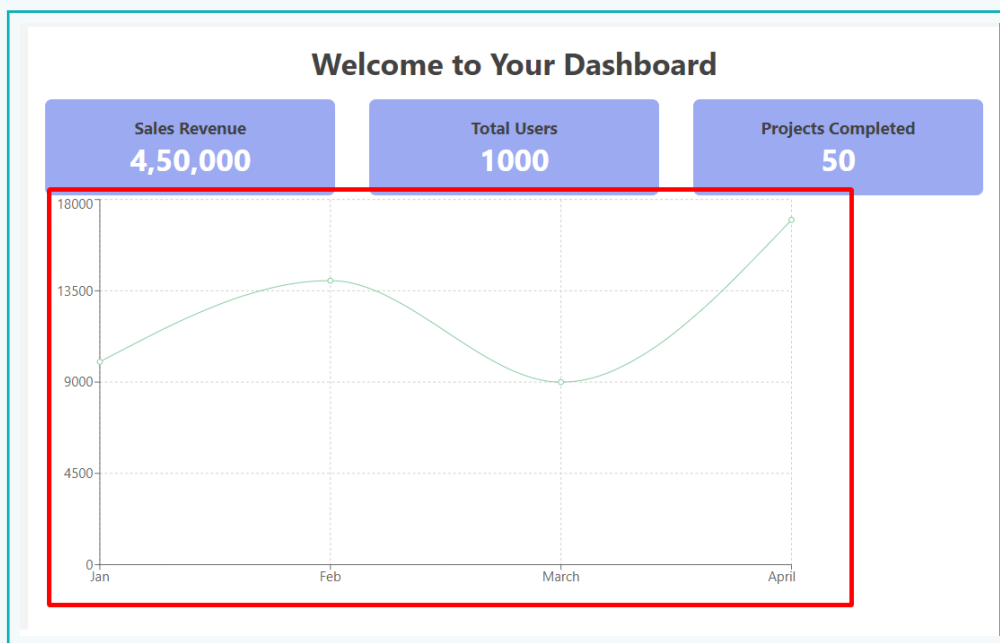
</main>
```

Explanation of the above components

- **<ResponsiveContainer>** - Creates a responsive container for displaying the Chart.
- **<LineChart data = {salesData} >** - Specifies that the Line Chart should use the data provided in the 'salesData' array.
- **<CartesianGrid />** - Creates the Grid layout with rows and columns. The 'strokeDasharray' specifies that the grid line will be dotted.
- **<XAxis datakey = "month" />** - Displays the 'month' property from the 'salesData' array along the X-axis.
- **<YAxis />** - Automatically determine and display the values along the Y-axis based on the provided data.
- **<Line type="monotone" dataKey="amt" stroke="#82ca9d"/>** - The 'monotone' type specifies that the line should be smooth. 'dataKey' informs the <Line> component which field to use for plotting the line (in this case, 'amt'), and 'stroke' defines the colour of the line (in this case, a shade of green).

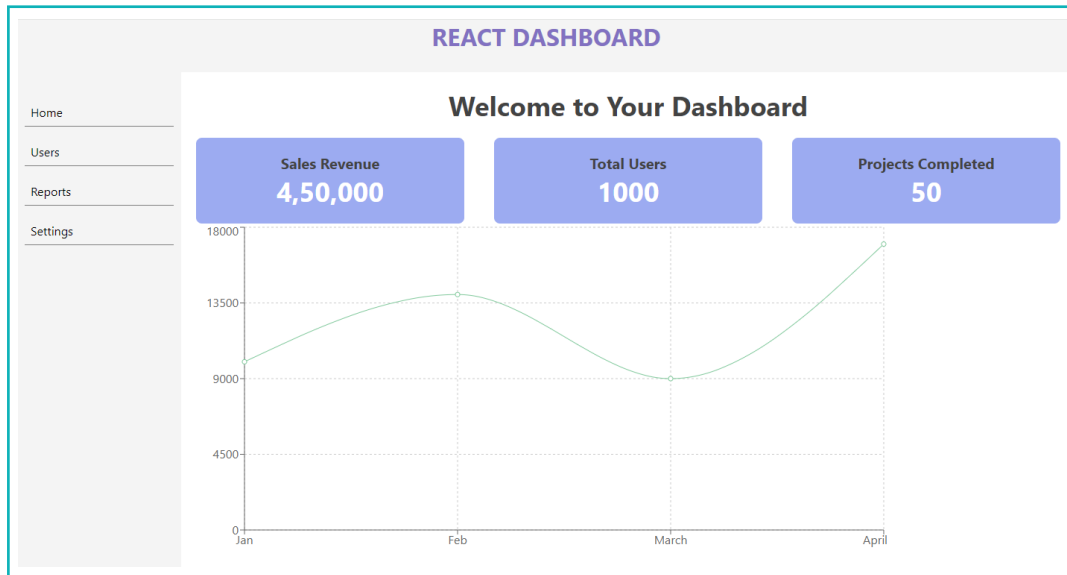
Activity

- 8 Check the generated chart and look for any errors.



The chart should be generated correctly and display the data from the *salesData* array.

Output -



In this lesson, we learned how to showcase data using widgets and charts. We created widgets to display data on the Dashboard. We incorporated the 'Recharts' library to generate charts on the Dashboard.

Exercise

State if the statements are True or False.

1 We can use the 'Ctrl + C' key combination to start a React application.

▲ _____

2 Arrays are represented using square brackets ([]).

▲ _____

Exercise

Fill in the Blanks.

- 3 The _____ component contains the 'data' attribute.
- 4 The 'strokeDasharray' attribute belongs to the _____ component.
- 5 The _____ attribute within the `<Line>` component defines the colour of the line.

Answer the following questions in one line.

- 6 What is a Charting Library?

Tech Flash

- | | |
|-----------------|---|
| Widgets | : It refers to small rectangular or square-shaped elements used to display or highlight key information, show real-time updates, or allow user interaction. |
| Recharts | : It is a popular charting library designed specifically for React applications. It allows us to create customisable and interactive data visualisations easily. |
| Chart.js | : It is a popular JavaScript library for creating interactive charts in an easy manner. It is not bound to React and can be used with applications built using other technologies. It supports a wide range of chart types such as Line Chart, Bar Chart, Pie Chart, Scatter Plot, etc. |